

Day 1 — AI Accuracy Engineering, Statistics and PoC Diagnosis

In-depth explanation of every concept, with analogies,
worked examples and delivery guidance

Written so that you can read it once and teach from it confidently across a full eight-hour day. Each concept has: what it means in plain terms, why it matters, the technical detail behind it, an analogy for the room, a worked example where numbers help, and the misunderstandings participants typically arrive with.

Enterprise AI Engineering Bootcamp · Companion to the 67-slide Day 1 presentation

How to Use This Document

Read end to end once before Day 1. Keep it beside you during delivery — sections map to slide numbers in order. Four kinds of box appear throughout:

Analogy. A comparison outside AI that makes the idea land immediately. Use when you see blank faces.

How to say it. Wording shaped for precision. Use directly when the point matters.

Watch for. The misunderstanding participants arrive with, and how to correct it without embarrassing anyone.

Deeper detail. Technical background for questions from the stronger participants. Not needed for the main delivery.

The audience is experienced engineers who have built AI proofs of concept. They know the terms. What they usually lack is the discipline connecting them — which is why Day 1 is a loop (measure, diagnose, hypothesise, prove) rather than a list of techniques.

Timing guide

Slot	Content	Notes on pacing
09:00 – 11:00	Module 1, sections 1.1 to 1.18	Dense. Do not rush 1.8 to 1.16 — the cost, calibration and threshold material is the practical core
11:00 – 11:45	Lab 1	Walk the room continuously; push for concrete examples
11:45 – 13:00	Module 2A, sections 2.1 to 2.10	2.9 and 2.10 (intervals, bootstrap) often change behaviour most
13:45 – 15:15	Module 2B, sections 2.11 to 2.25	Keep test selection brisk; spend the time on power and multiplicity
15:15 – 16:15	Lab 2	Challenge every team on the power calculation
16:15 – 17:15	Module 3, sections 3.1 to 3.12	Energy is low at this hour — lean on the whiteboard techniques in 3.6
17:15 – 18:00	Lab 3 and assignment brief	Protect this; it produces the assignment input

Module 1 — From Model Accuracy to System Reliability

1.1 Why AI proofs of concept fail in production

A proof of concept answers one question: *is this possible at all?* A production system answers a different one: *is this dependable enough to build a business process on?* Nearly every failure follows from confusing the two.

Cause	What actually happens
Distribution shift	The demo used data the team curated — clean, complete, representative of expectations. Production data arrives with missing fields, new formats, new variants, and changes gradually. The model was never asked to handle any of it.
No definition of done	"Make it more accurate" has no end state. Without a named metric and agreed threshold, the project cannot be completed — only funded until patience runs out.
Last 10% is 90% of the work	A convincing demo takes weeks. Edge cases, error handling, monitoring, access control, integration and support take quarters.
No failure ownership	When the system is wrong, nothing captures it. No queue, no record, no feedback path. The system cannot improve and trust erodes with every unlogged mistake.
Integration debt	The PoC ran in a notebook against a CSV. Production needs authentication, rate limits, retries, audit logs and a rollback path — none designed.
Unowned metric drift	The business definition of success shifts but the measurement does not. The team optimises a number nobody uses any more.

Analogy — the concept car. A concept car at a motor show drives beautifully across a polished stage for ninety seconds. That tells you nothing about whether it starts at minus ten, survives a pothole, or can be serviced by a dealer. A PoC is the motor show. Production is the road.

How to say it. "Your PoC was built to prove the idea is possible. That was the right thing to build. The problem is that nothing about proving possibility tells you how to make it dependable — those are different engineering problems, and the second is why we are here."

1.2 The seven dimensions of the gap

Use this table to justify the structure of the whole week. Data, scale, users, failure handling, change, observability and accountability all differ between a PoC and production, and each is a separate workstream. Teams that budget only for the model are surprised by six of the seven.

Point out the mapping: Day 2 addresses architecture and scale, Day 3 observability, Day 4 change management through MLOps, Day 5 accountability and governance.

1.3 Demo accuracy versus production accuracy

Two different numbers on two different populations. Demo accuracy is measured on data the team chose, answering questions the team knew about, judged by people who wanted it to work. Production accuracy is measured on whatever arrives, answering whatever is asked, judged by people who will stop using it the moment it wastes their time.

Watch for. Someone will say "our PoC gets 85% so we only need 10% more." That framing treats 85% as a property of the system. It is a property of the test that was run. The first useful question is not "how do we get to 95%" but "on what

population was 85% measured, and does that population resemble production?" Ask gently — the answer is often that nobody knows.

1.4 Accuracy, reliability and trustworthiness

Property	Question it answers	Belongs to	Improved by
Accuracy	How often is the output correct?	The model	Modelling work
Reliability	Correct consistently, under real conditions, over time?	The system	Engineering work
Trustworthiness	Will a user act on the output?	The relationship	Design work

Analogy — the weather forecast. A forecast can be accurate and useless. If it says "70% chance of rain" but is right only half the time it says 70%, you stop trusting it. And with no indication of confidence at all, you cannot decide whether to cancel the outdoor event. Accuracy alone does not tell you whether to act.

1.5 Service level indicators and objectives

An SLI is what you measure. An SLO is the target you commit to. Borrowed directly from site reliability engineering, and it transfers cleanly to AI systems.

Layer	Example indicator	Example objective
Quality	Proportion of answers judged correct on review	$\geq 92\%$ over a rolling 30 days
Safety	Rate of policy violations or unsafe outputs	$\leq 0.1\%$ of requests
Coverage	Proportion answered without escalation	$\geq 70\%$ automated
Latency	95th percentile end-to-end response time	≤ 4 seconds
Cost	Cost per successful resolution	$\leq ₹12$ per query
Availability	Successful responses divided by total requests	$\geq 99.5\%$

How to say it. "A system that is accurate, slow, expensive and unavailable has still failed. Your metric set needs at least four of these layers, not just the first."

1.6 and 1.7 Metric selection

This is the intellectual heart of the client's brief. They asked to move systems from roughly 60% to 95%. The request cannot be met as stated, because "95%" specifies nothing until you say *95% of what*.

Work through the use-case table on the slide, then give the six-question selection framework: what decision does the output drive; which error is more expensive; how rare is the positive class; is confidence used downstream; does timing matter; what does the user actually experience.

How to say it — handling the 95% question directly. "The 95% target is a reasonable business instinct and a poor engineering specification. Not because it is too ambitious, but because it does not say what to measure. For a maintenance system, 95% accuracy is trivially achievable and worthless. For a defect inspector it might mean shipping five defective parts per hundred. Our first job is turning that instinct into something you can engineer against."

1.8 and 1.9 The cost of errors, and finding the optimal threshold

Two ways to be wrong, almost never equally expensive. The equation that decides the threshold:

$$\text{Total cost} = (\text{false positives} \times \text{cost per false positive}) + (\text{false negatives} \times \text{cost per false negative})$$

Both counts change as the threshold moves. Sweep it, compute total cost at each point, take the minimum.

Worked example

False reject costs ₹400 in scrapped material. False accept costs ₹40,000 in warranty and rework. Ratio 1:100.

Threshold	False rejects /1,000	False accepts /1,000	Reject cost	Accept cost	Total
0.50	30	4	₹12,000	₹160,000	₹172,000
0.35	62	2	₹24,800	₹80,000	₹104,800
0.25	95	1	₹38,000	₹40,000	₹78,000
0.20	140	1	₹56,000	₹40,000	₹96,000
0.10	310	1	₹124,000	₹40,000	₹164,000

Illustrative. The shape is the point: cost falls, reaches a minimum, rises again. The optimum here is 0.25 — a 55% cost reduction from moving one number, with no retraining.

How to say it. "Nobody retrained a model to get from ₹172,000 to ₹78,000. Somebody asked finance what each mistake costs and moved a number. That is the cheapest improvement available to most systems in this room."

Watch for. Teams will say they cannot get cost figures. Push back gently: an order of magnitude is enough. Knowing the ratio is 1:100 rather than 1:1 changes the threshold decisively; knowing whether it is 1:80 or 1:120 barely matters.

1.10 Aleatoric and epistemic uncertainty

Two fundamentally different reasons a model is unsure, and they call for opposite responses.

- **Aleatoric** — irreducible noise in the data itself. Two genuinely identical inputs produce different outcomes. A borderline surface mark two inspectors would label differently. More data does not reduce it.
- **Epistemic** — the model has not seen enough of this kind of case. A product variant that appeared twice in training. Targeted data does reduce it.

Deeper detail. Ensembles separate the two reasonably well: disagreement *between* ensemble members indicates epistemic uncertainty, while consistent uncertainty *within* every member indicates aleatoric. Bayesian neural networks and Monte Carlo dropout aim at the same decomposition more formally. For most industrial work, a small ensemble is sufficient and far simpler.

How to say it. "Before funding a data collection programme, ask which kind of uncertainty you have. If it is aleatoric, more data will not move the number and you will spend six months proving it."

1.11 Where confidence scores come from

Model type	Confidence source	Typical behaviour
Logistic regression	Predicted probability	Reasonably calibrated by construction
Gradient-boosted trees	Predicted probability	Usually over-confident near 0 and 1
Deep neural network	Softmax output	Systematically, often badly over-confident

Object detector	Objectness × class score	Poorly calibrated; varies sharply with object size
Large language model	Token log-probabilities, or self-report	Self-reported confidence is close to meaningless
Ensemble	Variance across members	Better calibrated; captures epistemic uncertainty

Deeper detail — why neural networks are over-confident. Training minimises cross-entropy loss, which is minimised by pushing predicted probabilities towards 0 and 1. The network is rewarded for being decisive, not for being honest about its uncertainty. Modern architectures with high capacity can drive training loss very low while learning nothing about how uncertain they should be. This is why temperature scaling — a single parameter that softens the logits — recovers so much calibration so cheaply.

1.12 and 1.13 Calibration

Definition: a model is calibrated if, among all cases where it says 80% confident, it is correct about 80% of the time.

Checking it — the reliability diagram

1. Bucket every prediction by stated confidence — 0–10%, 10–20%, and so on.
2. For each bucket, compute the proportion actually correct.
3. Plot stated confidence on x against observed accuracy on y.
4. Perfect calibration is the diagonal. Below it is over-confident; above is under-confident.

Expected Calibration Error summarises this in one number: the average gap between stated and observed, weighted by bucket population. Below about 0.05 is usable for threshold design. Above 0.15, confidence-based routing will misbehave badly.

Fixing it

Method	How it works	When to use
Platt scaling	Logistic regression fitted on the model's scores	Binary problems, small calibration sets
Isotonic regression	Non-decreasing step function fitted to scores	More flexible; needs more data; can overfit
Temperature scaling	Divides logits by one learned constant	Neural networks. Simple, preserves ranking, very effective
Ensembling	Averages across several models	Improves calibration and captures epistemic uncertainty
Conformal prediction	Produces sets with guaranteed coverage	When a formal guarantee is required

Analogy — the colleague who is always certain. Everyone has worked with someone who says "I'm sure" whether they are right or wrong. You learn to ignore their confidence entirely, because it carries no information. An uncalibrated model is that colleague — and every downstream rule that keys off confidence is listening to them.

Watch for. Calibration must be fitted on a separate held-out calibration set — never on training data, never on the final test set. Teams sometimes calibrate on test data and then report test performance, which is a subtle form of leakage.

1.14 Choosing the operating threshold

Method	How it works	When
--------	--------------	------

Cost minimisation	Sweep the threshold, compute expected cost, take the minimum	The default when both error costs are known
Constraint satisfaction	Fix one metric at a required level, maximise the other	When a regulatory or safety floor exists
F-beta optimisation	Maximise a weighted harmonic mean of precision and recall	When you want one tunable number
Youden's J	Maximise sensitivity + specificity - 1	Balanced problems with no cost information

Whichever you use: set the threshold on a validation split, confirm on the held-out test set, never tune it on test data.

1.15 to 1.17 Three-band design, queue sizing and escalation

The most useful reframe of Module 1: **a system does not need to be right every time — it needs to know when it might be wrong**. Two thresholds create three bands: act automatically, route to human review, decline and escalate.

Sizing the review queue — the arithmetic teams skip

Input	Example	Source
Daily decision volume	4,000 inspections	The process, not the model
Deferral rate at chosen thresholds	12%	The confidence distribution
Reviews generated per day	480	Volume × deferral rate
Time per review	90 seconds	Measure it, do not estimate
Reviewer hours needed	12 hours	Reviews × time
Available capacity	8 hours	The binding constraint

Demand exceeds capacity by 50%. Three options: narrow the band and accept more automated error, add capacity, or improve the model until the deferral rate fits. That is a real design decision, and it is invisible unless you do this arithmetic.

Escalation design — five rules

1. **Route by reason, not just score.** Low confidence from an unfamiliar variant needs a different destination from low confidence caused by a poor image.
2. **Carry context forward.** The human needs the input, the top candidates, the confidence, and what the system found ambiguous. A bare "unknown" wastes their time.
3. **Set a response expectation.** An escalation with no service commitment becomes a queue nobody works.
4. **Close the loop.** Every resolved escalation is a labelled example. If it does not flow back, you are discarding your most valuable data.
5. **Monitor the escalation rate.** A rising rate is an early drift signal, often visible before accuracy metrics move.

Analogy — the experienced technician. A good technician does not diagnose every fault themselves. They handle what they know, ask a colleague about the unfamiliar, and escalate anything involving safety. Nobody calls that a weakness. It is what makes them dependable — and it is exactly the behaviour to design into the system.

Watch for. Participants treat "route to a human" as failure and try to engineer it away. Reframe: the review queue is a component, not a workaround. It makes the system deployable at today's accuracy instead of in six months, and it produces exactly the labelled data needed to improve.

1.18 Error budgets

Decide in advance how much failure is acceptable, state it as a number, manage against it. Example: "no more than two missed critical defects per 10,000 inspections per month."

Three benefits: it converts an open argument into an agreed figure; it tells you when to stop improving; and it tells you when to stop shipping features and fix reliability instead.

Burn rate is the operational refinement. Track how fast the allowance is consumed. Burning 50% of a monthly budget in week one is an alert in itself, well before exhaustion — and it triggers a change freeze and diagnostic review.

Lab 1 — Facilitation notes

Nine categories: data, labelling, retrieval, model, prompt, architecture, security, workflow, evaluation errors. For each — does it apply, one concrete example witnessed, rough frequency.

Facilitation. Teams write category names and stop. Push for specifics at every table. "Model errors" teaches nothing; "the model confuses variant A with variant B when the label is partly obscured" is a finding you can act on. If time allows, have one team also run the queue-sizing arithmetic from 1.16 — it usually produces a visible reaction.

Module 2A — Evaluation Foundations

2.1 The evaluation dataset comes first

Six properties of a set worth having: representative, stratified, held out and untouched, contains the hard cases, versioned, owned.

Analogy — the driving test. If the examiner only tested you on an empty car park at midday, your pass would predict nothing about driving in traffic in the rain. The test has to contain the difficult conditions or the certificate is meaningless.

How to say it. "Show of hands — who has an evaluation set that is representative, stratified, never used for tuning, contains your known hard cases, is versioned, and has a named owner?" Usually two or three hands. "Then that is the highest-value work available to you before anything else."

2.2 Sampling strategies

Strategy	How it works	When
Simple random	Equal probability for every item	Balanced classes, high volume
Stratified	Sample within each class to fixed proportions	The right default for AI evaluation
Cluster	Sample whole groups — a machine, a site	Items within a group are correlated
Importance / oversampling	Deliberately over-sample rare critical cases	Rare failure modes random sampling misses
Time-based	Sample from a specific, later window	Anywhere drift is plausible
Adversarial / targeted	Deliberately collect known-hard cases	Building the stress portion of a gold set

2.3 Splitting strategies

Split	Description	Risk
Random row	Shuffle and divide	Leaks when rows correlate — same machine in train and test
Stratified	Preserve class balance	Safe default for balanced tabular problems
Group	All rows for one entity in one split	Essential when rows share a machine, part or document
Temporal	Train on past, test on future	The only honest split for time-dependent problems
k-fold CV	Rotate the held-out fold	Good for small data; still needs group or time awareness
Nested CV	Inner loop tunes, outer evaluates	Prevents tuning leakage during hyperparameter selection

How to say it. "For anything with a time dimension, a random split will flatter your model and mislead you completely. It lets the model see the future. Every predictive maintenance PoC in this room should be split temporally, and I would be surprised if all of them are."

2.4 Building a gold set

1. **Define what correct means** — written, with examples of correct, incorrect and borderline. Owned by the business.
2. **Choose 50–100 seed cases** — common cases, known failures, edge cases, each rare class.
3. **Experts answer independently** — two where feasible; their disagreement rate is your ceiling.
4. **Resolve disagreements explicitly** — the discussion produces the written rule. Record the rule.
5. **Record provenance and version** — which expert, which date, which definition version.
6. **Grow it from failures** — every fixed production failure becomes a permanent case.

Step 4 is where the value concentrates. The written rule is frequently more valuable than the labels themselves, because it makes all future labelling consistent.

2.5 The confusion matrix and derived metrics

	Predicted positive	Predicted negative
Actually positive	True positive	False negative — missed
Actually negative	False positive — false alarm	True negative

Metric	Formula	Plain meaning
Precision	$TP / (TP + FP)$	Of everything flagged, how much was right
Recall / sensitivity	$TP / (TP + FN)$	Of everything that mattered, how much was caught
Specificity	$TN / (TN + FP)$	Of the negatives, how many were correctly left alone
F1	$2PR / (P + R)$	Harmonic mean, equal weight to both
F-beta	weighted harmonic mean	β above 1 favours recall; below 1 favours precision

Analogy — fishing with a net. Precision is how much of your catch is fish rather than seaweed. Recall is how many of the fish in the lake you caught. A finer net raises recall and lowers precision. No net maximises both, and which you prefer depends on whether seaweed or missed fish costs more.

2.6 Metrics beyond precision and recall

Metric	What it adds	When
PR-AUC	Area under the precision-recall curve	Imbalanced problems — far more informative than ROC-AUC
Matthews correlation	Balanced measure using all four cells	A single robust number under imbalance
Balanced accuracy	Mean recall across classes	Quick imbalance-aware substitute for accuracy
Cohen's kappa	Agreement corrected for chance	Model versus human, or human versus human
Log loss	Penalises confident wrong answers	When probability quality matters
Brier score	Mean squared error of probabilities	Calibration-sensitive
Per-class recall	Recall broken out by class	Always. Aggregates hide the class you care about

2.7 Why ROC-AUC misleads under imbalance

The false positive rate uses the total number of negatives as its denominator. When negatives vastly outnumber positives, even a large number of false positives produces a tiny false positive rate — so the ROC curve looks excellent while the system flags mostly noise. Precision uses the number of predicted positives instead, so every false positive damages it directly.

Worked illustration

100,000 items, 100 of them defective. The model flags 1,000 items and catches 90 defects.

- False positive rate = $910 / 99,900 \approx 0.9\%$ — the ROC curve looks superb.
- Precision = $90 / 1,000 = 9\%$ — the inspector opening those thousand cases finds 910 are fine.

Both numbers are correct. Only one describes the experience of using the system.

2.8 Class imbalance

The single most useful example in Day 1. A machine fails one day in a thousand. A model that predicts "no failure" always, without looking at anything, is 99.9% accurate, has never predicted a failure, is completely worthless — and comfortably beats a 95% target.

Deliver this as a question first. "I have a model that predicts equipment failure with 99.9% accuracy. Would you ship it?" Wait. Some will say yes. Then reveal. The room's reaction to its own answer is what makes it stick — do not shortcut it by explaining first.

What to do instead

- Report recall on the rare class, never overall accuracy.
- Use precision-recall curves rather than ROC.
- Set the threshold from cost asymmetry.
- Stratify or oversample so rare cases appear in useful numbers.
- Consider class weighting, SMOTE or focal loss — but measure whether each actually helped.

Deeper detail — on SMOTE and resampling. Synthetic oversampling generates new minority examples by interpolating between existing ones. It frequently improves training-set metrics and frequently fails to improve production performance, because interpolated examples occupy regions of feature space that do not correspond to real cases. Class weighting in the loss function is usually a safer first move, and threshold adjustment is safer still since it changes no training at all. Whatever you try, measure on a real held-out set.

2.9 and 2.10 Confidence intervals and the bootstrap

A single number without an interval invites false conclusions. "Accuracy improved from 0.81 to 0.84" means nothing until you know the uncertainty on both.

Method	Use for	Note
Normal approximation	Large samples, proportions near 0.5	Breaks down near 0 or 1, and on small samples
Wilson interval	Proportions, including small samples	Sensible default for accuracy, precision, recall
Bootstrap	Any metric at all	Resample with replacement, recompute, take percentiles
Bayesian credible interval	When prior information exists	Useful for very small evaluation sets

Worked intuition

On 100 evaluation examples, an accuracy of 0.84 carries a 95% interval of roughly 0.75 to 0.90. An apparent improvement from 0.81 to 0.84 sits entirely inside that band — indistinguishable from noise. On 1,000 examples the interval narrows to about 0.82 to 0.86 and the comparison becomes meaningful.

The bootstrap in five steps

1. Draw N items from your evaluation set of size N, *with replacement*.
2. Recompute the metric on that resample.
3. Repeat 1,000 to 10,000 times.
4. You now have a distribution of the metric.
5. The 2.5th and 97.5th percentiles give a 95% confidence interval.

It works for metrics with no simple formula — mean average precision, NDCG, faithfulness scores, end-to-end task success. For a paired comparison, bootstrap the *difference* between the two systems on the same resampled items and check whether the interval contains zero.

How to say it. "Twenty lines of code, works for any metric, makes no distributional assumptions. If you take one statistical technique from today, take this one."

Module 2B — Hypothesis Formulation and Testing

2.11 Three levels of hypothesis

Level	Question	Example
Business	What outcome improves?	"Service engineers will find the right procedure faster."
Technical	What change causes it?	"Hybrid retrieval with reranking returns more relevant passages."
Measurable	What number moves, by how much, measured how, on what data?	"Context recall on the 80-question gold set v3 rises from 0.61 to at least 0.75."

Watch for. Teams write "improve retrieval quality" and believe they have a hypothesis. Ask three questions: measured how, on what data, and what result would make you abandon this? If they cannot answer all three, it is an intention, not a hypothesis.

2.12 Null and alternative hypotheses

- **Null (H_0)** — the assumption of no effect. You assume it is true and look for evidence strong enough to reject it.
- **Alternative (H_1)** — the claim you are making. You never prove it directly; you reject H_0 or fail to.

Analogy — presumption of innocence. A court does not set out to prove innocence. It assumes it, and asks whether the evidence is strong enough to overturn the assumption. If not, the verdict is "not proven" — not "proven innocent". Failing to reject the null is the same: you have not shown your change works, which differs from showing it does not.

One-sided or two-sided? Two-sided by default. A one-sided test is only defensible if a change in the opposite direction would genuinely not interest you — rarely true, because a change that makes things worse is important information.

2.13 and 2.14 Test selection and assumptions

Situation	Example	Test
Two proportions, independent	Pass rates of two separate groups	Two-proportion z-test, or chi-square
Two means, independent	Average latency of two systems	Two-sample t-test (Welch's by default)
Same items, continuous	Both models scored on the same gold set	Paired t-test
Same items, pass/fail	Both models right or wrong per item	McNemar's test
Non-normal or small samples	Skewed scores, n below about 30	Mann-Whitney U, Wilcoxon signed-rank
Categorical association	Do failures cluster by product family?	Chi-square test of independence
Three or more variants	Three retrieval strategies	ANOVA or Kruskal-Wallis, plus multiplicity correction
No assumption you trust	Any metric, any shape	Bootstrap or permutation test

Assumptions and what breaks them

Test	Key assumptions	Common violation
------	-----------------	------------------

t-test	Roughly normal sampling distribution; independence	Skewed metrics; correlated items from one machine
Paired t-test	Differences roughly normal; pairs matched	Pairs not actually the same items
Chi-square	Expected count ≥ 5 per cell	Sparse cells — use Fisher's exact instead
McNemar	Paired binary; adequate discordant pairs	Too few cases where the systems disagree
Mann-Whitney	Independence; similar distribution shapes	Very different shapes
ANOVA	Normality; equal variances	Unequal variances — use Welch's ANOVA

Why paired designs matter in AI work. Evaluation items differ in difficulty. Some questions are genuinely hard and both systems fail; some are easy and both succeed. An unpaired comparison treats all that variation as noise, and the noise is often larger than the improvement you are looking for. Pairing compares item by item, so item difficulty cancels and smaller real effects become detectable with the same amount of data.

2.15 McNemar's test

When both systems are evaluated on the same items and each item is simply right or wrong, McNemar is the correct test — and that is the situation almost every time you compare two models.

	System B right	System B wrong
System A right	a — both right	b — A right, B wrong
System A wrong	c — A wrong, B right	d — both wrong

Cells a and d carry no information — both systems agreed. Only the discordant cells b and c matter, and the test asks whether they differ more than chance allows.

Practical point worth making. "If b plus c is small — the two systems rarely disagree — then no test will find a difference, and that is itself the answer. Your change did very little. You have learned something useful without needing a p-value at all."

2.16 Permutation tests

1. Compute the observed difference between the two systems.
2. Pool the results and randomly reassign each item's outcome between the two systems.
3. Recompute the difference on the shuffled assignment.
4. Repeat 10,000 times to build the null distribution.
5. The p-value is the proportion of shuffled differences at least as extreme as the observed one.

No distributional assumption, works for any metric, and easy to explain to a sceptical stakeholder — you are asking how often chance alone would produce a difference this large.

2.17 and 2.18 Type I/II errors and power

Error	You concluded	Consequence
Type I	The change helped — when it did not	You ship complexity that costs money for no benefit, and trust a method that does not work. Controlled by the significance level, commonly 0.05.
Type II	The change did not help — when it did	You discard a real improvement, usually due to an under-sized evaluation set. Controlled by power, target 0.80.

Power analysis — the four linked quantities

Significance level, power, effect size and sample size. Fix any three and the fourth is determined. In practice you fix the first three and solve for sample size.

Improvement to detect	Approximate n per group	Note
0.60 → 0.80 (20 points)	~90	Large effects are cheap to confirm
0.60 → 0.70 (10 points)	~360	Four times the sample for half the effect
0.60 → 0.65 (5 points)	~1,500	Again roughly four times

At significance 0.05 and power 0.80, two-sided. Figures are approximate and intended to convey the scaling relationship, not to substitute for a calculation.

The rule to write on the whiteboard. "Halving the effect you want to detect roughly quadruples the sample you need. That is why an evaluation set of thirty examples can only ever confirm large, obvious wins — and why so many good ideas get discarded as 'no difference'."

2.19 Effect size

Measure	What it expresses	Interpretation
Absolute difference	Raw change in the metric	Most honest for business conversation: 0.61 → 0.75 is +14 points
Relative difference	Change as a percentage of baseline	+23% — persuasive, but inflates small absolute changes
Cohen's d	Difference in standard deviations	0.2 small, 0.5 medium, 0.8 large
Odds ratio	Multiplicative change in odds	Common in classification and risk contexts
Number needed	Items required for one extra success	Very intuitive for operational stakeholders

Analogy — the weighing scale. Statistical significance tells you the scale is reliable enough to trust the reading. Effect size tells you whether the difference it shows is worth caring about. A scale precise enough to detect two grams in a shipment of steel is working perfectly and telling you nothing useful.

2.20 Multiple comparisons

Test twenty ideas at a 0.05 threshold and you should expect one false positive even if none work.

Correction	How it works	Trade-off
Bonferroni	Divide the threshold by the number of tests	Very conservative; loses power quickly
Holm-Bonferroni	Sequentially rejective version	Uniformly more powerful; a good default
Benjamini-Hochberg	Controls expected proportion of false discoveries	Best when screening many candidates
Pre-registration	Decide the primary metric and test in advance	Not a correction — a discipline. The strongest defence

How this appears in practice. A team evaluates a change against fifteen metrics, finds two that improved significantly, and reports those two. Nothing dishonest was intended, but the result is close to meaningless. The fix costs nothing: name one

primary metric before running the experiment.

2.21 and 2.22 A/B testing, and the peeking problem

Stage	What it is	When
Offline evaluation	Both systems on the same gold set	Always first. Filters most bad ideas cheaply
Shadow deployment	Challenger runs live, output not shown	Real behaviour, latency and cost at zero risk
A/B test	Traffic split, outcomes compared	Strongest evidence; needs volume and a stopping rule
Staged rollout	Gradual promotion with rollback	How the change reaches production safely

Peeking. Checking results repeatedly and stopping when they look favourable dramatically inflates false positives — with frequent enough checking the rate can rise from 5% to above 30%. Fix the sample size and stopping rule in advance, or use a sequential design that accounts for planned interim looks.

The same problem in offline evaluation. Running your change against the gold set, adjusting, re-running, adjusting again, and reporting the best result is peeking by another name. Each pass consumes a little of the test set's honesty. This is precisely why the test set is held out and the validation set is where you iterate.

2.23 Label quality and inter-annotator agreement

Principle: where two qualified experts disagree, no model can learn a consistent boundary, because the boundary does not exist in the data. Your realistic ceiling is roughly the level at which humans agree with each other.

Kappa	Interpretation
Below 0.20	Poor — the task definition itself needs work before modelling continues
0.21 – 0.40	Fair — expect a low ceiling
0.41 – 0.60	Moderate — usable but the definition would repay attention
0.61 – 0.80	Substantial — usable ground truth
Above 0.80	Excellent — rare in genuinely subjective tasks

Interpretation bands are conventional guidance, not a standard. Treat them as orientation.

How to say it. "If two of your inspectors agree only 70% of the time, a 95% model target is not ambitious — it is incoherent. You would be asking the model to be more consistent than the ground truth it learns from."

2.24 and 2.25 Bias, variance, calibration and drift

Concept	Plain explanation and diagnostic signature
Bias	Systematically wrong in the same direction. Too simple for the problem, or trained on unrepresentative data. Signature: poor on both train and test.
Variance	Unstable — small training changes cause large behavioural changes. Signature: excellent on train, poor on test.
Drift	The world changed after training. Signature: performance decays over time while nothing in the code changed.
Calibration	Stated confidence does not match observed accuracy. Signature: threshold rules behave unexpectedly.

Three kinds of drift

Type	What changed	Detection	Response
Covariate shift	Input distribution	Compare feature distributions over time	Retraining on recent data often suffices
Label shift	Class balance	Monitor predicted class proportions	Recalibrate thresholds; check the process
Concept drift	Input-to-outcome relationship	Performance falls while inputs look unchanged	Retraining required; old labels may not apply

Detection methods: Population Stability Index on key features, Kolmogorov-Smirnov tests between reference and current windows, monitoring the confidence distribution, tracking the escalation rate, and most reliably a small continuously refreshed labelled sample scored on a schedule.

Deeper detail — why covariate shift matters operationally. It is the only kind detectable *without labels*. In production, labels arrive late — sometimes weeks late, sometimes never. Input distribution monitoring gives you a signal on the same day. It will not tell you performance dropped, but it tells you to go and check.

Lab 2 — Facilitation notes

Three hypotheses, each with: the measurable claim, the null hypothesis, the chosen test and why, the assumptions and whether they hold, the pass/fail criterion set in advance, the dataset and size, a power calculation, and what happens if it fails.

Facilitation. Push hardest on two things. First, the pass/fail criterion — if a team cannot say what result would make them abandon the change, it is not a hypothesis. Second, the power calculation — most teams discover their planned evaluation set cannot detect the effect they hope for, and that discovery is the most valuable output of the lab.

Module 3 — Error Taxonomy and Root-Cause Analysis

3.1 Why more data is usually not the answer

If the real problem is...	Then more data...
Ambiguous labels	adds more ambiguity. The boundary does not exist yet.
Domain shift	from the old distribution does not help. You need the right data, not more.
Poor feature representation	cannot compensate for a signal the inputs do not contain.
Evaluation leakage	tells you nothing, because the measurement is already dishonest.
Aleatoric uncertainty	cannot reduce it by definition. You are at the task ceiling.
Under-represented cases	does help — targeted collection of specific conditions, not bulk collection.

3.2 Data-centric versus model-centric

Model-centric holds the data fixed and iterates on architecture, hyperparameters, loss and ensembling. It is where the literature focuses and where engineers instinctively start. On a stalled industrial PoC, returns are usually small because the constraint is rarely the model.

Data-centric holds the model fixed and iterates on label consistency, taxonomy definition, coverage of edge cases and targeted collection. Less glamorous, and usually where the large gains are.

A diagnostic worth giving them. "Freeze the model entirely. Fix only the labelling of your worst-performing class. Re-measure. If performance moves substantially, your constraint is data, not modelling — and you now know where to spend the next quarter."

3.3 The usual root causes

Cause	What it looks like in an industrial setting
Ambiguous labels	Two inspectors classify the same borderline surface mark differently. Ground truth is unstable.
Weak ground truth	Maintenance logs record when a repair was <i>booked</i> , not when degradation <i>began</i> .
Poor feature representation	The sensor that would reveal the fault is not fitted, or is sampled too coarsely.
Domain shift	Training images under controlled lighting; production under changing shift lighting with glare and oil film.
Edge cases and long tail	Rare situations that each look negligible and together dominate failures.
Measurement leakage	Information from the future, or from the test set, has entered training.

3.4 A workflow for error analysis

1. **Sample failures** — 100 to 200 actual failures, random rather than cherry-picked.
2. **Look at them** — individually. This cannot be automated and is where the insight comes from.
3. **Propose categories** — let them emerge from the data rather than imposing a taxonomy first.
4. **Classify and count** — counting turns impressions into priorities.
5. **Weight by cost** — frequency alone misleads. Multiply by business impact.

6. **Estimate headroom** — how much would fixing each category entirely gain? This ranks the work.

7. **Act on the top two** — not the top ten. Depth beats breadth.

Watch for. Step 2 resists automation and teams resist doing it. Insist. An afternoon spent actually looking at two hundred failures reliably beats a month of hyperparameter tuning, and everyone who has done it once never skips it again.

3.5 Slice-based analysis

Slice by	Why it reveals problems	Typical finding
Class or defect type	Rare classes are invisible in the aggregate	Two classes carry 80% of errors
Product family or variant	Models learn the majority variant	One variant far below the mean
Time period	Drift and seasonality hide in the average	Decline after a process change
Data source or site	Sensors, cameras and operators differ	One plant behaves differently
Input difficulty	Small objects, low contrast, long documents	Small-object recall half the headline
Confidence band	Performance should decline smoothly with confidence	A misbehaving band reveals miscalibration

Build slice reporting into the evaluation harness once and every future experiment gets it free.

3.6 Root-cause techniques

Three standard quality-engineering tools that transfer directly. An industrial audience already knows all three — the point is that they apply to AI systems too, which is a good rapport moment.

- **Five whys.** The model missed the defect → the image was too dark → the lighting rig was moved → nobody owns lighting configuration. The fourth answer is the actionable one.
- **Fishbone (Ishikawa).** Group causes by category — data, labels, model, process, environment, measurement. Prevents the group converging on the first plausible explanation.
- **Pareto analysis.** Rank causes by contribution, plot the cumulative total. Usually two or three categories dominate. This turns a taxonomy into a work plan.

3.7 and 3.8 Leakage

Type	What happens	Prevention
Temporal	Features computed across the whole series, including after the prediction point	Compute causally — only data available at prediction time
Target	An input that is a consequence of the outcome	Trace every feature to when it becomes known
Group	The same machine or document in train and test	Split by group, not by row
Tuning	The test set used repeatedly to select settings	Keep a final held-out set used once
Preprocessing	Scaling or imputation fitted before splitting	Fit transformations inside the training fold only
Duplicate	Near-identical records across splits	De-duplicate before splitting, including near-duplicates

Analogy — the exam paper. If a student has seen the paper beforehand, their score tells you nothing about how well they know the subject. Leakage is the model having seen the paper. The score is real; it just does not measure what you think it measures.

The seven-point checklist

1. Does any feature encode the outcome?
2. Was the split random on time-dependent data?
3. Do groups span the split?
4. Was preprocessing fitted before splitting?
5. Are there near-duplicates across splits?
6. How many times has the test set been used?
7. Does performance hold on a later time window?

Rule of thumb worth giving them. "If your offline results look surprisingly good, assume leakage until you have ruled it out. Being pleasantly surprised by a model is usually a warning, not a success."

3.9 Turning failures into permanent tests

In software engineering, a fixed bug leaves behind a test so it cannot silently return. AI systems need exactly the same discipline and almost never have it.

Suite	Contents	Purpose
Regression cases	Every fixed failure, permanently	Prevents silent reintroduction
Capability cases	Things the system must always get right	Defines the floor below which you cannot ship
Stress cases	Ambiguous, adversarial, malformed inputs	Reveals the boundary of competence
Safety cases	Inputs that must never produce certain outputs	Non-negotiable; a failure blocks release
Slice suites	Grouped by class, variant, site, condition	Enables slice reporting on every run

Analogy — the maintenance log. No engineering organisation fixes a recurring machine fault without recording it. The log is what turns individual repairs into a pattern, and the pattern leads to the design change. A failure taxonomy is the maintenance log for an AI system.

3.10 Ablation studies

When you change four things and performance improves, you have learned almost nothing. An ablation isolates each component's contribution by removing it and re-measuring.

Configuration	Context recall	Delta	Verdict
Full pipeline	0.78	—	Baseline
Remove reranking	0.71	-0.07	Substantial — keep
Remove hybrid search	0.74	-0.04	Moderate — keep
Remove query rewriting	0.775	-0.005	Within noise — remove and simplify
Remove metadata filter	0.62	-0.16	Largest single contributor

Two valuable outcomes: you learn where to invest, and you find components adding complexity without value. Removing query rewriting here makes the system simpler, cheaper and faster at no measurable cost.

3.11 The hypothesis backlog

The output of error analysis is not a report. It is a ranked, visible list of experiments.

Hypothesis	Expected gain	Confidence	Effort	Priority
Fix labelling of classes C and D	+0.08 recall	High	2 weeks	High
Add metadata filtering to retrieval	+0.06 recall	High	3 days	Highest
Augment low-light training images	+0.04 recall	Medium	1 week	Medium
Try a larger backbone model	+0.02 recall	Low	2 weeks	Lowest

Score as $\text{expected gain} \times \text{confidence} \div \text{effort}$. It need not be precise — it needs to be explicit, so ranking is a shared decision rather than whoever argues hardest. Note that the largest model change ranks last: high effort, low confidence, small expected gain. That is the typical pattern.

3.12 The improvement loop

Collect → classify → prioritise → write a test → hypothesise → experiment → decide.

Steps 4 and 7 are the ones teams skip, and they are the two that make the loop compound rather than repeat. Without step 4, fixed failures return. Without step 7 — a recorded decision and its reasoning — next year's team repeats this year's experiments.

Lab 3 and the assignment

Lab 3: classify 100 real failures, count, weight by cost, estimate headroom, identify the top three levers with a hypothesis for each. Have team members classify independently first — where they disagree is itself a finding, and it makes section 2.23 concrete rather than theoretical.

The assignment has seven deliverables: baseline and metric set, failure taxonomy, root-cause analysis, threshold and cost analysis, three hypotheses, experiment design per hypothesis, and the completed leakage checklist.

Closing the Day

1. A single accuracy number is not a specification.
2. You cannot improve what you cannot measure honestly.
3. State the hypothesis and pass criterion before the experiment — and check you have the power to detect it.
4. Find the failure pattern before reaching for more data.
5. Threshold and calibration work is nearly free, and often the largest single gain available.

Close by asking each team to name the single biggest thing they learned about their own PoC today. It surfaces honest findings and sets up Day 2 — because most of what they name will turn out to be an architecture problem.

Companion documents: **Day 1 Presentation** (67 slides with speaker notes) and **Day 1 Question Bank** (anticipated participant questions with model answers).